AI AGGREGATOR V2 — План разработки

Проект: ~/Desktop/ai-aggregator-v2

Дата старта: 17 марта 2026

Стек: Next.js 14 + TypeScript + Tailwind + Zustand + ReactFlow + Vercel AI SDK

ФИЛОСОФИЯ ПРОДУКТА

> "Мы делаем не AI, который пишет сценарии за человека.
> Мы делаем живую кинематографическую среду, где человек учится писать,
> чувствует сцену и работает с AI-командой как с творческими партнёрами."

- AI не заменяет автора — помогает думать, усиливать, замечать
- Продукт ощущается как режиссёрская комната, не как офисный софт
- Канва — мозг проекта. Упрощённые страницы — другие представления тех же данных
- Поток письма священен — нельзя ломать попапами, панелями, спамом AI

АРХИТЕКТУРА: ДВА РЕЖИМА

Режим 1: КАНВА (Board)

Бесконечный холст на ReactFlow. Все элементы проекта — ноды:

- Сценарий (ScriptDocNode) — лист A4
- Раскадровка (StoryboardNode) — карточки шотов
- Генерации (ImageGenNode, VideoGenNode)
- AI-чат (AIChatNode)
- Заметки, референсы, медиа

Режим 2: ФОКУС-СТРАНИЦЫ (Focus Layers)

Overlay поверх канвы. Один стор данных, разные представления:

- Script Write — полноэкранный редактор сценария
- Storyboard — карточки сцен в ряд
- Timeline — горизонтальная шкала

Переход между режимами — плавная анимация (лист из канвы "раскрывается").

ЭТАПЫ РАЗРАБОТКИ

ЭТАП 1: ФУНДАМЕНТ ✅ (выполнено)

Цель: чистая база, стартовая страница, канва

- [x] Новый проект Next.js + Tailwind + Zustand + lucide
- [x] Стартовая страница (KOZA, New/Old Project)
- [x] ReactFlow canvas с dot grid
- [x] Кнопка Back на канве
- [x] MiniMap
- [x] Светлая кремовая тема

ЭТАП 2: СКРИПТ-ДОКУМЕНТ НА КАНВЕ ✅ (выполнено)

Цель: нода-лист A4, переход в редактор

- [x] ScriptDocNode — лист A4 с плюсиком
- [x] Меню: New Script / Upload Script
- [x] Плавный переход канва → редактор (overlay)
- [x] Обратный переход редактор → канва
- [x] Титульная страница (название, автор, дата, Draft)
- [x] Шрифт PT Mono (кириллица + латиница)
- [x] Фиксированный тёмный фон, скроллируемые страницы

ЭТАП 3: РЕДАКТОР СЦЕНАРИЯ

Цель: перенести ScreenplayEditor из старого проекта и адаптировать

3.1 — Стор данных

- [ ] Создать `src/store/script.ts` — текст сценария, undo/redo
- [ ] Создать `src/store/project.ts` — шоты, метаданные проекта
- [ ] Данные титульника сохраняются в стор (название, автор, дата)

3.2 — Вторая страница (после титульника)

- [ ] После Enter на титульнике — появляется второй лист
- [ ] Второй лист = textarea для свободного письма
- [ ] Автоскролл к новому листу
- [ ] Автопагинация (когда текст переполняет лист — появляется следующий)

3.3 — Парсинг сценария

- [ ] Перенести `src/lib/screenplayFormat.ts` из старого проекта
- [ ] Автоопределение типов строк (scene heading, character, dialogue, action, transition, parenthetical)
- [ ] Подсветка синтаксиса (overlay поверх textarea, как в старом проекте)
- [ ] Горячие клавиши: Tab для переключения типов, Cmd+Shift+I для INT., и т.д.

3.4 — Floating toolbar

- [ ] При выделении текста — всплывающий тулбар
- [ ] Кнопки: Rewrite, Expand, Shorten, Fix Grammar
- [ ] Кнопки отправляют запрос в AI, результат показывается inline

3.5 — Звук печатной машинки

- [ ] Тихий клик при нажатии клавиш (Web Audio API)
- [ ] Переключатель вкл/выкл

ЭТАП 4: AI-АССИСТЕНТ

Цель: AI-помощник встроен в процесс письма

4.1 — Плавающий чат (уже начат)

- [ ] BoardAssistant работает поверх канвы
- [ ] Стриминг ответов через /api/chat
- [ ] Выбор модели (Claude/GPT/Gemini)
- [ ] Drag для перемещения окна

4.2 — Контекст сценария

- [ ] Ассистент получает текущий текст сценария в system prompt
- [ ] Ассистент знает текущую сцену, где стоит курсор
- [ ] Quick actions: Improve scene, Punch dialogue, Add detail

4.3 — Inline AI-команды (killer feature)

- [ ] Пользователь пишет команду прямо в тексте: "/перепиши жёстче"
- [ ] Система распознаёт команду (начинается с "/" или через триггер)
- [ ] AI обрабатывает, показывает diff/preview
- [ ] Пользователь принимает (Enter) или отклоняет (Esc)
- [ ] Команда исчезает, текст обновляется

4.4 — AI-роли (позже)

- [ ] Cinematographer — свет, кадр, композиция
- [ ] Dialogue Coach — диалоги

- [ ] Story Editor — драматургия
- [ ] Director's Assistant — общее видение
- [ ] Роли можно вкл/выкл
- [ ] Замечания появляются деликатно, не спамят

ЭТАП 5: АВТОМАТИЧЕСКОЕ РАЗБИЕНИЕ НА СЦЕНЫ

Цель: текст автоматически разбивается на сцены

5.1 — Scene engine

- [ ] По мере письма — автоматическое определение границ сцен
- [ ] Scene Heading (INT./EXT.) = начало новой сцены
- [ ] У каждой сцены: id, текст, описание, статус, порядок

5.2 — Scene list

- [ ] Боковая панель со списком сцен
- [ ] Клик на сцену — скролл к ней в редакторе
- [ ] Drag для перестановки сцен

5.3 — Scene ↔ Board sync

- [ ] Каждая сцена = нода на канве
- [ ] Изменения в редакторе отражаются на канве и наоборот

ЭТАП 6: AMBIENT VISUALS (мотивация через картинки)

Цель: на фоне появляются иллюстрации по тексту

6.1 — Генерация промптов

- [ ] Claude анализирует текущую сцену → генерирует промпт для картинки
- [ ] API роут `/api/ambient-prompt`

6.2 — Генерация картинок

- [ ] Imagen 4 Fast ($0.02/картинка) или DALL-E
- [ ] API роут `/api/ambient-image`
- [ ] Стиль: indie graphic novel, muted tones, noir

6.3 — Отображение

- [ ] Фоновый слой за текстом в режиме редактирования
- [ ] 1 картинка = на весь фон. 2-4 = комикс-сетка
- [ ] Интервал: каждые 90 сек, если текст изменился (50+ символов)
- [ ] Кнопка toggle вкл/выкл

ЭТАП 7: РАСКАДРОВКА (Storyboard)

Цель: из сцен создаются шоты с визуалами

7.1 — Shot cards

- [ ] Каждая сцена разбивается на шоты
- [ ] У шота: narrative, imagePrompt, videoPrompt, camera type, duration
- [ ] Карточки с превью сгенерированных картинок

7.2 — Storyboard view

- [ ] Фокус-страница: карточки шотов горизонтально
- [ ] Drag для перестановки
- [ ] Клик на шот → редактирование промптов

7.3 — Storyboard на канве

- [ ] StoryboardNode — группа шотов как нода на канве
- [ ] Связь сцена → стрибордт через edge

ЭТАП 8: АДАПТИВНАЯ МУЗЫКА

Цель: музыка генерируется в реальном времени через Tone.js

- [ ] Скорость набора → интенсивность музыки
- [ ] Claude определяет настроение сцены → тональность/инструменты
- [ ] Грусть = минорный pad, Напряжение = пульсирующий бас, Радость = мажор
- [ ] Кнопка toggle вкл/выкл

ЭТАП 9: TIMELINE

Цель: временная шкала проекта

- [ ] Горизонтальная шкала с шотами
- [ ] Длительность каждого шота
- [ ] Превью картинок/видео на таймлайне
- [ ] Воспроизведение как аниматик

ЭТАП 10: ГЕНЕРАЦИЯ МЕДИА

Цель: генерация картинок и видео из шотов

- [ ] Image generation: DALL-E 3, Imagen 4, Flux, Midjourney
- [ ] Video generation: Kling, Seedance, Sora
- [ ] Ноды генерации на канве
- [ ] Очередь генерации, статусы, превью

ЭТАП 11: ЭКСПОРТ И ДЕПЛОЙ

Цель: результат можно вынести из приложения

- [ ] Экспорт сценария в PDF (стандартный формат)
- [ ] Экспорт storyboard в PDF
- [ ] Экспорт видео-аниматика
- [ ] Supabase: регистрация, сохранение в облаке
- [ ] Деплой на Vercel

СТРУКТУРА ФАЙЛОВ (целевая)

```
ai-aggregator-v2/
├── src/
|   ├── app/
|   |   ├── page.tsx                    — Стартовая → Canvas
|   |   ├── layout.tsx                  — Шрифты, метадата
|   |   ├── globals.css
|   |   └── api/
|   |       ├── chat/route.ts           — Универсальный AI endpoint
|   |       ├── ambient-prompt/route.ts — Claude → промпт для картинки
|   |       └── ambient-image/route.ts  — Imagen/DALL-E → картинка
|   ├── components/
|   |   ├── board/
|   |   |   ├── Canvas.tsx              — ReactFlow обёртка
|   |   |   ├── BoardAssistant.tsx      — AI чат поверх канвы
|   |   |   └── nodes/
|   |   |       ├── ScriptDocNode.tsx   — Лист A4
|   |   |       ├── StoryboardNode.tsx  — Группа шотов
|   |   |       ├── ImageGenNode.tsx    — Генерация картинок
|   |   |       ├── VideoGenNode.tsx    — Генерация видео
|   |   |       ├── AIChatNode.tsx      — AI чат нода
|   |   |       ├── NoteNode.tsx        — Заметка
|   |   |       └── MediaNode.tsx       — Картинка/видео
|   |   ├── editor/
|   |   |   ├── ScriptWriterOverlay.tsx — Overlay редактора
|   |   |   ├── ScreenplayEditor.tsx    — Парсинг, подсветка, hotkeys
|   |   |   ├── TitlePage.tsx           — Титульная страница
```

```
│   │   │   ├── FloatingToolbar.tsx     — Тулбар при выделении
│   │   │   └── AmbientVisuals.tsx      — Фоновые иллюстрации
│   │   ├── storyboard/
│   │   │   ├── StoryboardView.tsx      — Фокус-страница раскадровки
│   │   │   └── ShotCard.tsx            — Карточка шота
│   │   ├── timeline/
│   │   │   └── TimelineView.tsx        — Фокус-страница таймлайна
│   │   └── ui/
│   │       └── ...                     — Общие UI компоненты
│   ├── store/
│   │   ├── script.ts                   — Текст сценария, undo/redo
│   │   ├── project.ts                  — Шоты, метаданные
│   │   ├── projects.ts                 — Список проектов
│   │   ├── board.ts                    — Тема, инструменты
│   │   └── assistant.ts                — Состояние AI ассистента
│   └── lib/
│       ├── models.ts                   — Все AI модели
│       ├── screenplayFormat.ts         — Парсинг сценария
│       └── sceneEngine.ts             — Определение сцен
├── .env.local                          — API ключи
├── package.json
└── tsconfig.json
```

ТЕКУЩИЙ СТАТУС

Выполнено:

- Стартовая страница ✅
- ReactFlow canvas с dot grid ✅
- ScriptDocNode (лист A4) ✅
- Переход канва ↔ редактор ✅
- Титульная страница ✅
- Шрифт PT Mono ✅
- AI SDK + API роут /api/chat ✅
- BoardAssistant (в процессе) 🔄

Следующий шаг:

- Этап 3: Редактор сценария (textarea на втором листе, парсинг, подсветка)

КАК РАБОТАТЬ С ЭТИМ ДОКУМЕНТОМ

1. Каждый новый чат с Claude начинай с: "Проект ~/Desktop/ai-aggregator-v2. Мы на этапе X. Задача: ..."
2. Прикладывай этот документ как контекст
3. Claude даёт ТЗ → ты копируешь в Claude VSCode → он выполняет
4. После выполнения — обновляй чекбоксы в этом документе